

Laboratorium 11 – MS SQL Server 2019

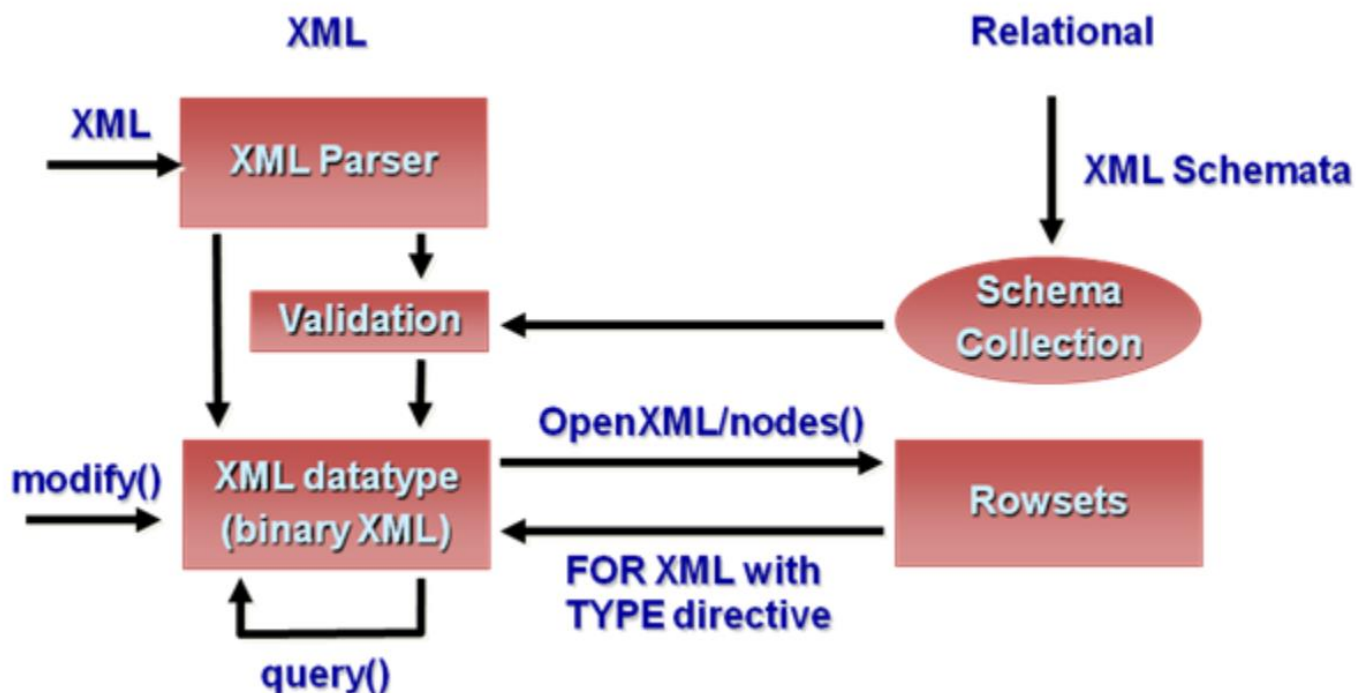
Temat: Przegląd możliwości SQL Server 2019 związanych z wsparciem dla XML

1. Wprowadzenie
2. Typ danych XML
3. Dane XML walidowane odpowiednim schematem XML Schema
4. Publikowanie dokumentów XML z danych relacyjnych
5. Przetwarzanie dokumentów XML i danych relacyjnych

1. Wprowadzenie

SQL Server 2000 oferował dwie funkcjonalności wspomagające tego typu rozwiązania: funkcję OPENXML do rozbijania dokumentów XML oraz klauzulę FOR XML instrukcji SELECT do ich składania.

Od wersji SQL Server 2005 i w następnych wersjach firma Microsoft wprowadziła nowe możliwości związane z przechowywaniem i przetwarzaniem danych XML. Ta nowa funkcjonalność ułatwia zapisywanie dokumentów w tym formacie przy zwiększonej wydajności w stosunku do wcześniej stosowanych technik. Przetwarzanie dokumentów XML realizowane jest przez wyspecjalizowane środowisko serwera oraz istnieje możliwość przetwarzania dokumentów XML i relacyjnych w oparciu o określony schemat działania.



Przetwarzanie dokumentu XML w MS SQL Server

Udostępnione funkcjonalności przetwarzania dokumentów XML w bazie MS SQL Server.

1. Dedykowany typ danych o nazwie XML, którego możemy użyć do przechowania całych dokumentów i do fragmentów.
2. Możliwość zarejestrowania schematów XML w SQL i przechowania zawartości schematów w bazie danych.
3. Automatyczna walidacja dokumentu z schematem, jeżeli istnieje.
4. Automatyczny podział dokumentu XML w celu zwiększenia efektywności przeszukiwania danych XML i zmiany zawartości takiego dokumentu.
5. Implementacja podzbioru specyfikacji języka W3C XQuery i XML-DML w zakresie przeszukiwania danych XML i zmiany zawartości takiego dokumentu.
6. Możliwość tworzenia indeksów XML podstawowych (primary) i indeksów drugorzędnych ułatwiających przetwarzanie dokumentów XML.
7. Wsparcie dla .NET Common Language Runtime (CLR) w SQL Server, które umożliwia procedurom składowanym przetwarzanie dokumentów XML w kodzie zarządzanym.

Model danych XML w MS SQL Server

Możliwości nowego typu danych ujawniają się dla danych reprezentujących, czy też raczej zawartych w kolekcjach obiektów typu **DataSet** w aplikacjach **.NET**. Usługi w takich aplikacjach dostarczają danych przenoszonych przez sieć przez protokół HTTP i mają postać strumienia tekstowego. To zapewnia, że mogą być przeszukiwane przez znane i standardowe metody i mogą dotrzeć wszędzie (HTTP z nie binarną zawartością). Jeżeli dołączy się możliwość rekonstrukcji zawartości DataSet po stronie klienta to staje się to bardzo atrakcyjne.

Do określenia typu danych w dokumencie XML służy schemat. Odbiorca na tej podstawie może określić ten typ i odpowiednio z niego skorzystać. Ten fakt jest podstawą modelu W3C **InfoSet** (link <http://www.w3.org/TR/xml-infoset/>).

Model ten rozważa dokumenty jako zestaw jednego lub wielu zestawów rekordów z określonymi typami danych czyli **typed rowsets**.

2. Typ danych XML

A. Typ danych XML

```
declare @xdoc xml
set @xdoc = '<?xml version="1.0"?> <test></test>'
select @xdoc
select cast(@xdoc as varchar)
```

B. Tworzymy tabele w bazie tempdb z kolumną XML

```
declare @xdoc xml
set @xdoc = '<xml/>'
print cast(@xdoc as varchar)
-- Outputs: "<xml/>"
create table SampleTable (RecordNo int, XDocument xml )
go
insert into SampleTable values (0, '<xml/>')
select * from SampleTable
```

C. Metody operowania na danych typu XML – `query()`, `value()`, `exist()`, `modify()` i `nodes()`.

Link do dokumentacji Microsoft: [https://learn.microsoft.com/en-us/previous-versions/sql/sql-server-2008-r2/ms190798\(v=sql.105\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/sql/sql-server-2008-r2/ms190798(v=sql.105)?redirectedfrom=MSDN)

- Metoda `query`(wyrażenie XQuery) – zwraca część dokumentu XML – wykorzystujemy wyrażenia XQuery lub XPath

```
declare @xdoc xml
set @xdoc = '<?xml version="1.0"?>
<test><jeden><nazwa/></jeden><dwa><nazwa/></dwa></test>'
select @xdoc.query('test/jeden')
```

- Metoda `value`(wyrażenie XQuery, typ) – pozwala na wyodrębnienie z typu danych XML wartości skalarnych (pobranie wartości o typie SQL). Parametr pierwszy XQuery lub XPath, drugi zwracany typ danych

```
declare @xdoc xml
set @xdoc = '<?xml version="1.0"?>
<test><jeden>Adam</jeden><dwa>Marek</dwa></test>'
select @xdoc.value(' (test/jeden) [1] ', 'varchar(20) ')
```

- Metoda `exist`(wyrażenie XQuery) – pozwala na sprawdzenie zawartości dokumentu XML pod kątem istnienia elementów lub atrybutów korzystających z wyrażenia XQuery

```
declare @xdoc xml
set @xdoc = '<?xml version="1.0"?>
<test><jeden>Adam</jeden><dwa>Marek</dwa></test>'
select @xdoc.exist('test/jeden')

set @xdoc = '<root PresentationDate="2006-11-02"/>'
select @xdoc.exist(
    '/root[(@PresentationDate cast as xs:date?) eq
    xs:date("2006-11-02")]')
```

```
set @xdoc = '<PresentationDate>2006-11-02</PresentationDate>'
select @xdoc.exist(
    '/PresentationDate[(text() [1] cast as xs:date ?) =
    xs:date("2006-11-02")]')
```

- Metoda `nodes`(wyrażenie XQuery) – stosowana do rozdzielenia danych XML na wiersze. Struktura metody: `nodes (XQuery) as Table(Column)`
wygenerowane wyniki relacyjne nie mogą być bezpośrednio pobrane. Można do nich uzyskać dostęp jedynie za pomocą metod `exist()`, `nodes()`, `query()` i `value()` typu danych xml lub sprawdzić je przy użyciu operatorów `IS NULL` i `IS NOT NULL`

```
declare @xdoc xml
set @xdoc = '<test><imie>Adam</imie><imie>Marek</imie></test>'
select T.c.query('.') results from @xdoc.nodes('test/imie') T(c)
```

• Metoda `modify(XMLdml)` – pozwala na modyfikację przechowywanego dokumentu XML. Przy użyciu tej metody można uaktualniać cały dokument XML lub tylko jego wybraną część. Modyfikacje dokumentu XML wykonujemy z wykorzystaniem języka XML DML (Data Manipulation Language).

Wyróżniamy trzy polecenia DML:

- `insert` – wstawianie węzłów XML,
- `delete` – usunięcie węzłów XML
- `replace value of` – zastępowanie zawartości węzła inną zawartością

```
declare @xdoc xml
-- wstawianie wezla po okreslonym wezle
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert <node>4</node> after (/test/node) [3]')
select @xdoc

-- wstawianie wezla przed okreslonym wezle
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert <node>0</node> before (/test/node) [1]')
select @xdoc

-- wstawianie wezla jako pierwszego w wezle
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert <node>0</node> as first into (/test) [1]')
select @xdoc

-- wstawianie wezla jako ostatniego w wezle
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert <node>4</node> as last into (/test) [1]')
select @xdoc

-- wstawianie wezla wewnatrz wezla
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert <node1>test1</node1> into (/test/node) [1]')
select @xdoc

-- dodawanie atrybutu
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('insert attribute val {"44"} into (/test/node) [2]')
select @xdoc

-- usuwanie wezla z dokumentu
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('delete /test/node[2]')
select @xdoc

-- usuwanie atrybutu z wezla
set @xdoc = '<test><node val="1">1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('delete /test/node[1]/@val')
select @xdoc
```

```
-- zmiana zawartosci wezla
set @xdoc = '<test><node>1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('replace value of (test/node)[2]/text()[1] with "22"')
select @xdoc

-- zmiana zawartosci atrybutu
set @xdoc = '<test><node
val="11">1</node><node>2</node><node>3</node></test>'
set @xdoc.modify('replace value of (/test/node/@val)[1] with "12"')
select @xdoc
```

3. Dane XML walidowane odpowiednim schematem XML Schema

W ramach bazy danych MS SQL Server można wprowadzać dane XML, które nie będą walidowane z wykorzystaniem XML Schema (untyped XML Data) i dane XML walidowane z zadany schematem XML Schema (typed XML Data). W ramach niniejszego punktu omówimy zagadnienie związane z danymi XML walidowanymi odpowiednimi danymi XML Schema.

A. W ramach interfejsu graficznego możemy sprawdzić obecne w bazie danych schematy wyszukując w odpowiedniej gałęzi bazy danych. Przykładowo dostępne schematy w bazie AdventureWorks2019.

Rozwijamy węzły:

AdventureWorks -> Programmability -> Types -> XML Schema Collection

Wykorzystując polecenie T-SQL w ramach konkretnej bazy danych:

```
SELECT * FROM sys.xml_schema_collections;
```

Przestrzenie nazw wykorzystane w ramach danych schematów wyświetlimy z wykorzystaniem następującego polecenia T-SQL.

```
SELECT * FROM sys.xml_schema_namespaces;
```

Postać schematu wyświetlana jest w jednej linii, czyli niełatwo poznać szczegóły takiego schematu. W celu przeglądania zawartości schematu w postaci bardziej czytelnej można skorzystać z poniższego polecenia.

```
SELECT xml_schema_namespace(N'relational_schema ',N'sql_identifier')
```

W ramach zapytania '**relational_schema**' jest schematem relacyjnym do którego należy XML Schema, a '**sql_identifier**' jest nazwą pod którą został on zapisany. Poniżej przykładowe polecenie dla jednego z schematów w bazie AdventureWorks2019.

```
SELECT
xml_schema_namespace(N'HumanResources',N'HRResumeSchemaCollection')
```

- B. Operacje tworzące i usuwające schemat można prześledzić z panelu graficznego. Wybieramy z drzewie obiektów bazy danych interesujący nas schemat a następnie wybierając prawym przyciskiem myszy menu kontekstowe i wybieramy polecenie Drop and Create to New Query Editor Windows. Otrzymamy poniższe polecenia.

```
DROP XML SCHEMA COLLECTION [relational_schema.]sql_identifier
```

```
CREATE XML SCHEMA COLLECTION [relational_schema.]sql_identifier AS'...'
```

Do modyfikacji dokumentu XML Schema wykorzystujemy polecenie – ALTER.

```
ALTER XML SCHEMA COLLECTION [ relational_schema.]sql_identifier ADD 'Schema Component'
```

[relational_schema.] - oznacza to opcjonalny element składni.

ALTER XML SCHEMA może dodawać nowy element na podstawowym poziomie, ale nie może modyfikować istniejących składowych (te są immutable). Metodą na zmianę jest zmieniienie kolumny XML na untyped, DROP i CREATE schematy, a następnie zmienić kolumny XML na typowane. Pociąga to za sobą także ponowne utworzenie indeksów (jeżeli były).

C. Tworzymy w bazie danych tempdb poniższy schemat ProductSchema.

```
CREATE XML SCHEMA COLLECTION ProductSchema AS '<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.microsoft.com/schemas/adventureworks2019/production"
xmlns:prod="http://www.microsoft.com/schemas/adventureworks2019/production">
<xs:element name="Product">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="prod:ProductID" />
      <xs:element ref="prod:ProductName" />
      <xs:element ref="prod:SupplierID" />
      <xs:element ref="prod:CategoryID" />
      <xs:element ref="prod:QuantityPerUnit" />
      <xs:element ref="prod:UnitPrice" />
      <xs:element ref="prod:UnitsInStock" />
      <xs:element ref="prod:UnitsOnOrder" />
      <xs:element ref="prod:ReorderLevel" />
      <xs:element ref="prod:Discontinued" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="ProductID" type="xs:integer" />
<xs:element name="ProductName" type="xs:string" />
<xs:element name="SupplierID" type="xs:integer" />
<xs:element name="CategoryID" type="xs:integer" />
<xs:element name="QuantityPerUnit" type="xs:string" />
<xs:element name="UnitPrice" type="xs:double" />
<xs:element name="UnitsInStock" type="xs:integer" />
<xs:element name="UnitsOnOrder" type="xs:integer" />
<xs:element name="ReorderLevel" type="xs:integer" />
<xs:element name="Discontinued" type="xs:boolean" />
</xs:schema>'
```

D. W bazie danych tempdb tworzymy tabelę dbo.ProductDocs z kolumną typem danych XML wykorzystując utworzony poprzednio schemat do walidacji danych.

```
CREATE TABLE ProductDocs (  
    ID INT IDENTITY PRIMARY KEY,  
    ProductDoc XML(ProductSchema) NOT NULL)
```

E. Dodajemy dane, jednocześnie sprawdzając działanie typowanej kolumny XML'a, czyli związanej ze schematem

```
INSERT INTO ProductDocs VALUES ('  
<Product xmlns="http://www.microsoft.com/schemas/adventureworks2019/production">  
    <ProductID>1</ProductID>  
    <SupplierID>1</SupplierID>  
    <CategoryID>1</CategoryID>  
    <QuantityPerUnit>10 opak. x 20 szt.</QuantityPerUnit>  
    <UnitPrice>18.0000</UnitPrice>  
    <UnitsInStock>39</UnitsInStock>  
    <UnitsOnOrder>0</UnitsOnOrder>  
    <ReorderLevel>10</ReorderLevel>  
    <Discontinued>0</Discontinued>  
    </Product>  
' )
```

Wprowadzając powyższy rekord otrzymamy komunikat z SQL Server'a podobny do poniższego. Powodem błędu jest brak elementu ProductName

```
--Msg 6965, Level 16, State 1, Line 1  
--XML Validation: Invalid content.  
  
--Expected  
element(s):http://www.microsoft.com/schemas/adventureworks/products:ProductName  
  
--where element 'http://www.microsoft.com/schemas/adventureworks/products:SupplierID'  
was specified.  
  
--Location: /*:Product[1]/*:SupplierID[1]
```

F. Następnie podajemy już poprawne dane

```
INSERT INTO ProductDocs VALUES ('  
<Product xmlns=" http://www.microsoft.com/schemas/adventureworks2019/production ">  
    <ProductID>1</ProductID>  
    <ProductName>Herbata</ProductName>  
    <SupplierID>1</SupplierID>  
    <CategoryID>1</CategoryID>  
    <QuantityPerUnit>10 opak. x 20 szt.</QuantityPerUnit>
```

```
<UnitPrice>18.0000</UnitPrice>
<UnitsInStock>39</UnitsInStock>
<UnitsOnOrder>0</UnitsOnOrder>
<ReorderLevel>10</ReorderLevel>
<Discontinued>0</Discontinued>
</Product>
')
```

G. Sprawdzenie poprawności wprowadzonych danych

```
SELECT * FROM ProductDocs
SELECT * FROM ProductDocs FOR XML AUTO
SELECT * FROM ProductDocs FOR XML RAW
```

4. Publikowanie dokumentów XML z danych relacyjnych

Tworzenie dokumentów XML w ramach bazy danych MS SQL Server wykonywane jest z wykorzystaniem polecenia **SELECT .. FROM ... FOR XML ...**.

Poniżej przedstawione zostaną przykłady wykorzystania polecenia do tworzenia dokumentów XML w bazie danych AdventureWorks2019.

W ramach ćwiczenia zapoznamy się z czterema formami polecenia FOR XML.

- a) **FOR XML RAW**
- b) **FOR XML AUTO**
- c) **FOR XML EXPLICIT**
- d) **FOR XML PATH**

A. Tworzenie dokumentu XML przy pomocy polecenia **FOR XML RAW**. Każdy wiersz zwracanego wyniku zamieniany jest na jeden element XML, a wartości poszczególnych kolumn na atrybuty.

```
SELECT * FROM sales.Customer FOR XML RAW
```

B. Tworzenie dokumentu XML przy pomocy polecenia **FOR XML AUTO, ROOT**. Klauzula **XML AUTO** działa podobnie jak **XML RAW**, jednak gdy zwracany wynik jest rezultatem złączenia wielu tabel, pokazuje on strukturę danych. Klauzula **ROOT("nazwa elementu")** – tworzy element główny. Klauzula ta nie może być użyta z klauzulą **XMLDATA** i **XMLSCHEMA**.

```
SELECT * FROM sales.Customer FOR XML AUTO, ROOT('customers')
```

C. Tworzenie dokumentu XML przy pomocy polecenia **FOR XML AUTO, ELEMENTS XSINIL**

Klauzula **ELEMENTS** w trybach **AUTO** i **RAW** powoduje, że wartości kolumn zostaną zamienione na elementy zamiast na atrybuty. Klauzula **XSNILL** – domyślnie pusty element jest pomijany. W przypadku wykorzystania z klauzuli **ELEMENTS** i **XSNILL** dołączony zostanie element pusty z atrybutem **xsi:nil="1"**.

```
SELECT * FROM sales.Customer FOR XML AUTO, ELEMENTS XSINIL,  
ROOT('customers')
```

D. Tworzenie odpowiedzi XML z wykorzystaniem konstrukcji **FOR XML PATH()** Format tworzego dokumentu XML jest określony przez wyrażenia XPath.

```
SELECT E.BusinessEntityID "@EmpID", FirstName "EmpName/First", MiddleName  
"EmpHumanResources.EmployeeName/Middle", LastName "EmpName/Last"  
FROM HumanResources.Employee E, Person.Person C  
WHERE E.BusinessEntityID = C.BusinessEntityID AND E.BusinessEntityID=218  
FOR XML PATH
```

```
SELECT SalesOrderID as "data() "  
FROM Sales.SalesOrderHeader S, Sales.Customer C, Person.Person P  
WHERE S.CustomerID = C.CustomerID AND C.CustomerID = P.BusinessEntityID FOR XML  
PATH  
( '')
```

```
SELECT P.BusinessEntityID as "@CustomerID", FirstName as "@CustomerName",  
(SELECT SalesOrderID as "data() "  
FROM Sales.SalesOrderHeader S  
WHERE S.CustomerID = C.CustomerID  
FOR XML PATH ( '' )) as "@SalesOrderIDs"  
FROM Person.Person P, Sales.Customer C WHERE C.CustomerID = P.BusinessEntityID  
FOR XML PATH('SalesOrders')
```

E. Tworzenie zapytania z konstrukcją **FOR XML EXPLICIT**. W zapytaniu strukturę zwracanego wyniku należy zdefiniować z wykorzystaniem dodatkowych informacji. W ramach zwracanego dokumentu XML można tworzyć elementy, atrybutu i sekcje **CDATA**

```
SELECT  
1 AS Tag,  
0 AS Parent,  
BusinessEntityID as [Contact!1!ContactID],  
FirstName as [Contact!1!FirstName!ELEMENT],  
MiddleName as [Contact!1!MiddleName!ELEMENTXSINIL],  
LastName as [Contact!1!LastName!ELEMENT],  
'<ContactInformation>Contact information</ContactInformation>' as  
[Contact!1!!cdata]  
FROM Person.Person  
WHERE BusinessEntityID=483 FOR XML EXPLICIT
```

F. Tworzenia zapytania z konstrukcją **FOR XML AUTO, TYPE**

Rezultat zapytania będzie zmienną typu XML, zamiast zmienną tekstową. Pozwala to na pisanie złożonych zapytań przy użyciu trybów **RAW** i **AUTO**

```
SELECT E.JobTitle,E.Birthdate
FROM HumanResources.Employee E
FOR XML AUTO, TYPE, ELEMENTS
```

```
SELECT BusinessEntityID, FirstName, LastName,
( SELECT E.JobTitle,E.Birthdate
FROM HumanResources.Employee E
FOR XML AUTO, TYPE, ELEMENTS)
FROM Person.Person P WHERE BusinessEntityID > 20770 ORDER BY BusinessEntityID FOR
XML AUTO, TYPE
```

G. Tworzenie **XML Schema** w zapytaniu **SELECT**.

Do wynikowego XML'a zostanie dołączony schemat XSD opisujące dane.

```
SELECT P.BusinessEntityID, P.FirstName, P.LastName
FROM Person.Person P
WHERE P.BusinessEntityID = 20000 FOR XML AUTO, XMLSCHEMA
```

```
SELECT P.BusinessEntityID,S.SalesOrderID, S.CustomerID,P.FirstName,P.MiddleName
FROM Sales.SalesOrderHeader S, Person.Person P, Sales.Customer C
WHERE S.CustomerID = C.CustomerID AND C.PersonID = P.BusinessEntityID
AND P.BusinessEntityID = 20000
FOR XML RAW, XMLSCHEMA, ELEMENTS XSINIL
```

Więcej przykładów można znaleźć pod adresem:

<https://www.red-gate.com/simple-talk/databases/sql-server/learn/using-the-for-xml-clause-to-return-query-results-as-xml/>

//baza danych AdventureWorks

5. Przetwarzanie dokumentów XML i danych relacyjnych

W ramach tego ćwiczenia wykorzystamy metody typu danych XML nie tylko do danych zawartych w polach XML bazy danych ale także utworzonych w poleceniu SQL przy pomocy klauzuli **FOR XML**.

Ćwiczenie realizujemy w bazie danych AdventureWorks2019.

A. Odpytywanie tabeli z polem XML

```
SELECT * FROM Sales.Store WHERE BusinessEntityID = 1000
```

B. Odpytywanie pola XML z wykorzystaniem metody **query** i polecenia **XPAT**

```
SELECT TOP 10 Demographics.query('declare default element namespace
"http://schemas.microsoft.com/sqlserver/2004/07/adventure-works/IndividualSurvey";
/IndividualSurvey/YearlyIncome')
FROM Sales.vIndividualCustomer
```

```
SELECT TOP 10 Demographics.value('declare default element namespace
"http://schemas.microsoft.com/sqlserver/2004/07/adventure-works/IndividualSurvey";
(/IndividualSurvey/YearlyIncome)[1]', 'varchar(250)')
FROM Sales.vIndividualCustomer
```

C. Sprawdzamy zawartość tabel Person.Address i HumanResources.Employee w bazie danych AdventureWorks2019

```
SELECT * FROM Person.Address
SELECT * FROM HumanResources.Employee
```

D. Tworzymy zmienną XML, umieszczamy w niej wynik działania polecenia **SELECT** z wyrażeniem **FOR XML**

```
DECLARE @x XML
SET @x = (SELECT AddressID, AddressLine1, City, StateProvinceID, PostalCode,
cast(SpatialLocation as nvarchar(MAX)) FROM Person.Address WHERE AddressID = 1
FOR XML AUTO, ELEMENTS, TYPE)
SELECT @x
```

E. Wyszukanie określonych danych w dokumencie XML metoda query()

```
DECLARE @x XML
SET @x = (SELECT TOP 50 City FROM Person.Address
FOR XML AUTO, ELEMENTS, TYPE)
SELECT @x.query('
<Cities> {
    for $city in /Person.Address/City
    return $city
} </Cities>
')
```

F. Wykorzystanie zagnieżdżonych poleceń SELECT do odpytywania bazy danych

```
DECLARE @x XML
SET @x = (
SELECT E.*, BA.AddressID, (
SELECT AddressID, AddressLine1, City, StateProvinceID, PostalCode,
cast(SpatialLocation as nvarchar(MAX)) FROM Person.Address Address
WHERE BA.AddressID = Address.AddressID
FOR XML AUTO, ELEMENTS, TYPE
) AS Addresses
FROM (HumanResources.Employee AS E
INNER JOIN Person.BusinessEntityAddress AS BA
ON E.BusinessEntityID = BA.BusinessEntityID ) INNER JOIN Person.Address AS PA ON
PA.AddressID = BA.AddressID
WHERE E.BusinessEntityID = 1
FOR XML AUTO, ELEMENTS, TYPE
)
SELECT @x
```